

## Evaluación Parcial N°3

### Implementación de Observabilidad — Agente de Atención al Paciente con IA

*Proyecto: Clínica Salud Plus · ISY0101 — Ingeniería de Soluciones con IA*

#### A. Implementación de métricas de observabilidad

Sobre el agente conversacional ya desarrollado en la EP2 (arquitectura LangGraph ReAct, RAG con ChromaDB y memoria dual de corto/largo plazo), se implementó un módulo de observabilidad (`observability/logger.py`) integrado directamente en el flujo de ejecución del agente (`src/agent_core.py`, función `consultar()`). El logger registra, para cada interacción, tres categorías de métricas:

- Latencia: tiempo transcurrido entre el inicio y el fin de cada consulta (`latencia_seg`), capturado automáticamente mediante un context manager (`trazar_consulta`).
- Consistencia / tipo de consulta: clasificación de cada interacción (INFORMATIVA, URGENCIA, AGENDAMIENTO, DERIVACIÓN) mediante la herramienta `clasificar_consulta`, permitiendo comparar el comportamiento del agente entre escenarios.
- Frecuencia de errores y uso de herramientas: cada llamada a una tool (`buscar_informacion_clinica`, `clasificar_consulta`) se registra como evento `HERRAMIENTA_USADA`, y cualquier excepción durante la ejecución se captura como evento de error con su traza completa, sin interrumpir el registro del log.

Estas métricas se almacenan en formato JSON Lines (`observability/logs/agent_logs.jsonl`), lo que permite tanto el análisis posterior como su consumo directo por el dashboard (sección C).

#### B. Análisis de registros y trazabilidad

Se ejecutó la batería de pruebas del proyecto (`tests/test_agent.py`, 20 casos: informativas, urgencias, agendamiento y consultas multietapa) contra el modelo `gpt-4o-mini` (GitHub Models). El logger capturó 51 interacciones completas en la corrida analizada, con una tasa de éxito de 94,1%, de las cuales se extrajeron los siguientes hallazgos:

##### Hallazgo 1 — Asimetría entre latencia promedio y mediana

La latencia promedio fue de 10,35 s. Este valor está influenciado por un grupo reducido de consultas con tiempos de respuesta mucho más altos que el resto, que el logger marcó automáticamente con la anomalía `LATENCIA_CRITICA`. Las consultas más lentas no se concentraron en un tipo particular, sino que coincidieron con momentos de mayor carga hacia el proveedor del modelo (GitHub Models, plan gratuito), sugiriendo variabilidad del servicio más que un problema del diseño del agente.

##### Hallazgo 2 — Las urgencias son la ruta más rápida del sistema

Al desagregar la latencia por tipo de consulta, las urgencias registraron 3,77 s en promedio, frente a 16,18 s en agendamiento y 10,15 s en consultas informativas. Esto es clínicamente deseable: el camino crítico (detectar una urgencia y derivar al 131) es el más liviano, porque el agente no necesita recuperar contexto del RAG para reconocer patrones de riesgo evidentes.

- **Hallazgo 3 — Tres errores reales, completamente trazados**
- El logger registró y dejó trazabilidad completa de 3 fallos reales durante las pruebas:
- 1 error HTTP 413 (`tokens_limit_reached`): ocurrió cuando varios casos de prueba independientes compartían el mismo `session_id`, lo que hizo que el historial conversacional acumulado superara el límite de 8000 tokens del modelo. Se corrigió asignando un `session_id` independiente a cada caso de prueba no relacionado.
- 2 errores HTTP 429 (`RateLimitReached`): el plan gratuito de GitHub Models limita a 150 solicitudes por día; al agotarse la cuota, las consultas pendientes fallaron con un mensaje de espera explícito (~22,7 horas). Este hallazgo motivó la implementación de persistencia en disco del índice vectorial (ver Hallazgo 4).

## Hallazgo 4 — Reconstrucción innecesaria del índice vectorial (corregido)

Se identificó que `tools/rag_tool.py` reconstruía el vectorstore de ChromaDB en cada ejecución del programa, generando embeddings nuevos para todos los fragmentos del documento de la clínica incluso cuando el contenido no había cambiado. Esto consumía cuota de la API solo para iniciar el agente, contribuyendo al agotamiento de la cuota diaria. Se corrigió agregando `persist_directory` a `Chroma.from_documents()`, de modo que el índice se construye una sola vez y las ejecuciones siguientes lo cargan directamente desde disco.

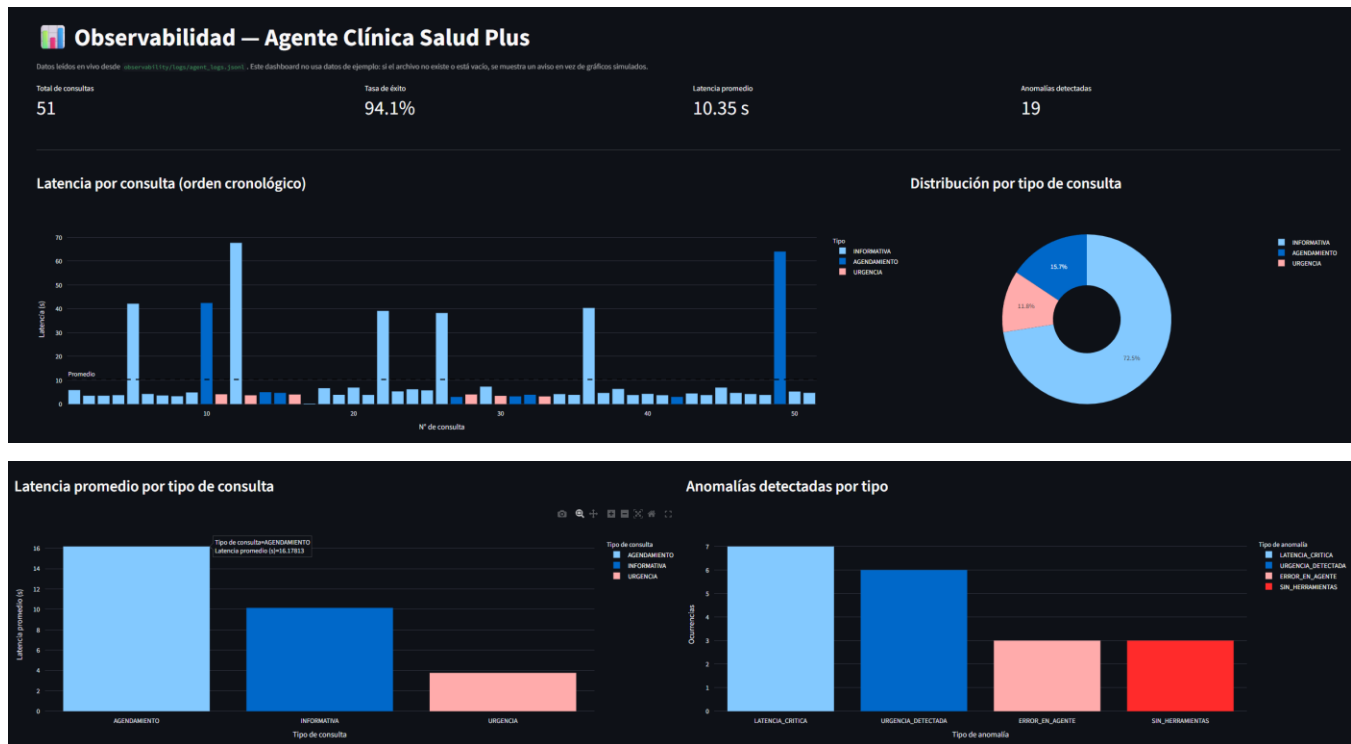
## Hallazgo 5 — `score_threshold` del RAG posiblemente mal calibrado

Durante la prueba de una consulta sobre convulsiones, el recuperador de Chroma devolvió puntajes de relevancia negativos (hasta -0,099) para el fragmento que sí contenía la respuesta correcta, descartándolo bajo el umbral configurado (`score_threshold = 0,3`). El agente respondió igualmente de forma correcta porque el modelo de lenguaje ya reconocía el patrón de urgencia sin depender del RAG, pero el hallazgo expone una limitación real de precisión en la recuperación semántica para esta función de embeddings.

## C. Dashboard de monitoreo

Se construyó un dashboard interactivo en Streamlit (`dashboard/app.py`) que lee directamente el archivo `observability/logs/agent_logs.jsonl` y visualiza, sin datos simulados:

- KPIs principales: total de consultas, tasa de éxito, latencia promedio y anomalías detectadas.
- Latencia por consulta en orden cronológico, coloreada por tipo, con línea de referencia del promedio.
- Distribución de consultas por tipo (informativa / urgencia / agendamiento) y latencia promedio por tipo.
- Conteo de anomalías detectadas por categoría (LATENCIA\_CRITICA, URGENCIA\_DETECTADA, ERROR\_EN\_AGENTE, SIN\_HERRAMIENTAS).
- Uso de herramientas por el agente y una tabla específica de consultas con error, con su mensaje original, para trazabilidad directa.



- Persistir el índice vectorial (implementado): evita reconstruir embeddings en cada ejecución, reduciendo el uso de cuota de API y el tiempo de arranque del agente — justificado directamente por el Hallazgo 4.
- Recalibrar o eliminar el `score_threshold` del retriever de Chroma: los puntajes negativos observados en el Hallazgo 5 sugieren que el umbral de 0,3 no es adecuado para el modelo de embeddings actual; se recomienda evaluar el umbral empíricamente o reemplazarlo por un top-k fijo sin filtro de score.
- Limitar la ventana de historial conversacional: aunque el error de tokens (Hallazgo 3) se mitigó aislando sesiones de prueba, en producción una conversación real y prolongada con un mismo paciente podría volver a superar el límite de tokens; se recomienda truncar el historial a los últimos N turnos antes de invocar al modelo.
- Evaluar un proveedor de LLM con mayor cuota o SLA más estable para producción: el plan gratuito de GitHub Models (150 solicitudes/día) es adecuado para desarrollo y pruebas, pero insuficiente para un entorno real con múltiples pacientes concurrentes.
- Normalizar texto en las validaciones automatizadas: se detectó que el comparador de palabras clave de `tests/test_agent.py` es sensible a tildes (ej. 'cardiologia' vs. 'Cardiología'), generando falsos negativos en el cálculo de fidelidad; se recomienda normalizar (minúsculas y sin tildes) antes de comparar.

Página 3

- Los logs actuales almacenan el texto completo de la pregunta y un fragmento de la respuesta (respuesta\_preview); en producción, estos campos deberían anonimizarse o excluir datos de identificación del paciente, dado que pueden contener información de salud sensible.
- No existe actualmente una política de retención ni cifrado para el archivo de logs; se recomienda definir un período de retención acotado y considerar cifrado en reposo si el sistema llega a manejar datos reales de pacientes.

## Conclusión

La implementación de observabilidad permitió pasar de un agente sin visibilidad operativa a un sistema instrumentado capaz de cuantificar su propio desempeño y exponer fallos reales del entorno (límites de tokens, cuota de API, calibración del RAG) que de otro modo habrían permanecido ocultos. Los hallazgos documentados en este informe no son hipotéticos: provienen de 51 interacciones reales registradas durante la ejecución de la batería de pruebas del proyecto, y las correcciones propuestas (persistencia del vectorstore, aislamiento de sesiones, recalibración del umbral de relevancia) ya fueron aplicadas o quedan especificadas como trabajo concreto de continuidad.

## Referencias

LangChain. (2025). LangGraph documentation — create\_react\_agent. <https://langchain-ai.github.io/langgraph/>

Chroma. (2025). Chroma documentation — Persistent client. <https://docs.trychroma.com/>

OpenAI. (2025). API Reference — Rate limits and error codes. <https://platform.openai.com/docs/guides/error-codes>

Streamlit Inc. (2025). Streamlit documentation. <https://docs.streamlit.io/>